Qiancheng Fu

qf59@cornell.edu
https://github.com/qcfu-bu

Education

2019 – 2026 **Ph.D. Computer Science**, Boston University, (Thesis Advisor: Hongwei Xi)

2019 – 2021 **M.Sc. Computer Science**, Boston University

2015 – 2019 **B.Eng. Computer Science**, North China University of Technology

Research Interests

I am interested in the theory and practice of programming languages, type systems, and formal verification. My research focuses on developing novel programming languages and type systems to develop verify properties of concurrent and distributed systems. These days, I am also interested in how logics and formal methods can be applied to make AI systems more reliable and trustworthy.

Research Publications

Conference Proceedings

1 C. Zhang, Q. Fu, H. Ji, I. S. D. Valle, A. Silva, and M. Gaboardi, "Outrunning big kats: Efficient decision procedures for variants of gkat," in *Programming Languages and Systems*, R. Krebbers, Ed., Cham: Springer Nature Switzerland, 2026, pp. 392–423, ISBN: 978-3-032-22723-2.

2 Q. Fu, A. Das, and M. Gaboardi, "Probabilistic refinement session types," PLDI, vol. 9, New York, NY, USA: Association for Computing Machinery, Jun. 2025. DOI: 10.1145/3729317

3 Q. Fu and H. Xi, "A calculus of inductive linear constructions," in *Proceedings of the 8th ACM SIGPLAN International Workshop on Type-Driven Development*, ser. TyDe 2023, Seattle, WA, USA: Association for Computing Machinery, 2023, pp. 1–13, ISBN: 9798400702990. DOI: 10.1145/3609027.3609404

4 M. Lemay, Q. Fu, W. Blair, C. Zhang, and H. Xi, "A dependently typed language with dynamic equality," in *Proceedings of the 8th ACM SIGPLAN International Workshop on Type-Driven Development*, ser. TyDe 2023, Seattle, WA, USA: Association for Computing Machinery, 2023, pp. 44–57, ISBN: 9798400702990. DOI: 10.1145/3609027.3609407

Journal Articles

1 Q. Fu and H. Xi, "A two-level linear dependent type theory," *ACM Trans. Comput. Logic*, Jun. 2026, Just Accepted, ISSN: 1529-3785. DOI: 10.1145/3819821

2 Q. Fu, H. Xi, and A. Das, "Dependent session types for verified concurrent programming," 2025. arXiv: 2510.19129 [cs.PL]. URL: https://arxiv.org/abs/2510.19129

3 Y.-T. Sun et al., "Human motion transfer with 3d constraints and detail enhancement," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–12, 2022. DOI: 10.1109/TPAMI.2022.3201904

Research Projects

- **LeanDP**: OpenDP is a community effort to build a comprehensive suite of tools for differential privacy. While the algorithms implemented in OpenDP have been carefully audited by security experts, the mathematical proofs of its algorithms have not been formally verified. The LeanDP project aims to prove, beyond any doubt, that the OpenDP implementation of differentially private algorithms are correct and satisfy their claimed privacy guarantees. For this purpose, LeanDP encodes the core components of OpenDP faithfully in the Lean theorem prover to formally verify their privacy guarantees. The project is a collaboration with the OpenDP team and is work in progress.
- **PReST** (Accepted to PLDI25): We develop a novel theory of probabilistic refinement session types (PReST) to *symbolically* specify and reason about probabilistic message passing concurrent programs. The soundness of the PReST type system ensures that the probabilistic distributions specified in communication protocols are respected at runtime. Most surprisingly, probabilistic refinement types can even be used to statically verify parametric distributions such as the uniform distribution over $\{0, ..., k\}$. Using PReST we are able to specify probabilistic distributed protocols such as Leader Election, Bounded Retransmission and Crowd Forwarding. We implement a type checker in OCaml using Z3 and CVC5 to efficiently solve complex generated constraints.

  Artifact: `https://zenodo.org/records/15151161`
- **On-the-fly GKAT** (Accepted to ESOP26): The theory of Kleene Algebra with Tests (KAT) allows one to decide the equivalence of imperative programs using an elegant equational theory. The theory of Guarded Kleene Algebra with Tests (GKAT) promises to improve the performance of KAT by restricting its scope to the deterministic fragment of KAT. In practice, however, GKAT decision algorithms tend to be even slower than KAT algorithms due to the necessity of performing additional normalization steps. To solve the issue of using GKAT in practice, we introduce a novel *on-the-fly* algorithm for GKAT which performs bisimulation in a greedy manner and defers normalization to when it is absolutely necessary. We develop the `rust-gkat` tool in Rust. Through experiments, we show that our tool performs orders of magnitudes faster than existing KAT solvers while also using significantly less memory.

  Repository: `https://github.com/qcfu-bu/rust-gkat`
- **SStruct**: The Substructural Dependent Type Theory (SStruct) is a dependently typed language which unifies all substructural typing disciplines such as linear, affine, relevant, and unrestricted in a single framework. This is accomplished by parameterizing the type system over an abstract *sort-order* structure which naturally induces different contraction and weakening behaviors for types. We fully formalize the theory of SStruct in Lean4 and prove its soundness in an end-to-end manner. In particular, we prove that SStruct is consistent as a logical framework under the lens of the Curry-Howard correspondence. Moreover, using heap semantics, we show that programs extracted from SStruct run memory clean and never get stuck during evaluation despite freeing linear and affine data.

  Repository: `https://github.com/qcfu-bu/SStructTT`
- **TLL** (Accepted to TOCL): The Two-Level Linear (TLL) dependent type theory is a programming language which combines dependent types and linear types. TLL features full Martin-Löf style dependent types to precisely reason about linearly typed programs. We fully verify the theory of TLL in Coq. A prototype optimizing compiler is implemented. It emits safe C code that is memory clean without need of runtime garbage collection. We also extend the compiler with features such as dependent session types for concurrency and *sort-polymorphic schemes* to write code generically for both linear and unrestricted types.

  Repository: `https://github.com/qcfu-bu/TLL`

Research Projects (continued)

- **CILC** (Accepted to TyDe23): The Calculus of Inductive Linear Constructions (CILC) is a linear dependent type theory which fully formalizes the mechanism for defining linear dependent inductive types. Linear dependent inductive types allow for the language runtime to perform mutation on data which appears to be immutable from the user's perspective. The entire theory of CILC is formalized in Coq and proven to be sound. CILC is one of the few dependent inductive type systems ever formalized in a theorem prover and, to the best of our knowledge, is also the only formalized linear dependent inductive type system.

  Repository: `https://github.com/qcfu-bu/CLC`
- **MT/DE-Net** (Accepted to TPAMI22): We propose a pipeline for retargeting character motion in videos by using 2 novel generative adversarial networks MT-Net and DE-Net. First, MT-Net uses of the projection of 3D human models to maintain the structural integrity of character poses during motion transfer. Next, DE-Net enhances details in generated frames by reusing the details in real source frames. This essentially eliminates the need for generating reusable details from scratch. Extensive experiments show that our approach yields better results both qualitatively and quantitatively than the state-of-the-art methods.

  Video Demo: `https://www.youtube.com/watch?v=w70WvywXy1s`

Talks

fall 2022	**A Calculus of Linear Constructions**. Boston University POPV Seminar
summer 2023	**A Calculus of Inductive Linear Constructions**. TyDe 2023, Seattle, WA
fall 2023	**A Two-level Linear Dependent Type Theory**. Boston University POPV Seminar
	Linear Dependent Types in Practical Programming. Harvard University PL Seminar
fall 2024	**Probabilistic Refinement Session Types**. Boston University POPV Seminar
summer 2025	**Probabilistic Refinement Session Types**. PLDI 2025, Seoul, South Korea
fall 2025	**Probabilistic Refinement Session Types**. Cornell University PLDG Seminar

Work Experience

2026 – present	**Postdoctoral Researcher**. Cornell University.
2025 – 2026	**Research Fellow**. Boston University.
fall 2022	**Instructor**. Boston University, Principles of Programming Languages.
2020 – 2024	**Teaching Fellow**. Boston University, Principles of Programming Languages.
2018 – 2019	**Research Assistant**. Institute of Computing Technology, Chinese Academy of Sciences.

Skills

programming	Rust, C/C++/CUDA/LLVM, OCaml, Haskell, ATS
proof assistants	Lean4, Rocq (+Iris), Agda, EasyCrypt
SMT solvers	Z3, CVC5, Alt-ergo
misc.	Type systems (dependent, linear, session), formal analysis of programming languages, compiler construction and optimization.